

Name: Rahul Bhati, 2223416

Subject: High Performance Computing (21BTCS030)

Theory Assignment-1

Q1

a) *Why did the performance increase of microprocessors slow down after 2002?* <L1/CO1>

Answer:

After 2002, the performance increase of microprocessors slowed due to **limitations in clock frequency scaling**. The primary reasons were:

- **Power and thermal constraints:** Higher clock speeds led to exponential increases in power consumption and heat generation (known as the *power wall*).
- **Memory wall:** The gap between CPU and memory speeds widened, reducing effective performance gains.
- **Instruction-level parallelism (ILP) limits:** Most sequential code could not be further optimized automatically by compilers.

Hence, simply increasing clock frequency was no longer sustainable.

b) *What is the primary solution proposed for addressing the limitations of single-core processors, and why is this solution beneficial?* <L1/CO1>

Answer:

The primary solution is **multi-core processors**.

- **Reason:** By integrating multiple cores on a single chip, each core can execute tasks concurrently, leading to higher overall throughput without increasing clock speed.
 - **Benefits:**
 - Improved performance through parallelism.
 - Better energy efficiency.
 - Scalability for multi-threaded applications.
-

c) *Explain Flynn's taxonomy for classifying computer architectures and provide examples for each category.* <L1/CO1>

Answer:

Flynn's taxonomy classifies computer architectures based on the number of instruction streams and data streams:

Category Description		Example
SISD	Single Instruction, Single Data — sequential execution.	Traditional uniprocessor (e.g., old Intel Pentium).
SIMD	Single Instruction, Multiple Data — same instruction operates on multiple data elements simultaneously.	GPUs, Vector processors.
MISD	Multiple Instruction, Single Data — multiple instructions operate on the same data stream (rare).	Fault-tolerant systems, systolic arrays.
MIMD	Multiple Instruction, Multiple Data — multiple processors execute different instructions on different data.	Multi-core CPUs, HPC clusters.

Q2

a) Explain the difference between task parallelism and data parallelism in the context of writing parallel programs. <L1/CO1>

Answer:

- **Task Parallelism:** Different tasks or functions execute concurrently on different cores. Each thread performs a distinct operation.
 - *Example:* One thread reads data, another processes it, and a third writes output.
- **Data Parallelism:** The same operation is applied to different portions of data in parallel.
 - *Example:* Applying a filter to different sections of an image simultaneously.

b) Why do serial programs generally not benefit from multicore processors, and what are the two main approaches to making serial programs run in parallel? <L1/CO1>

Answer:

Serial programs execute instructions sequentially and cannot utilize multiple cores simultaneously.

Two main approaches to parallelization:

1. **Manual parallelization:** Modify the code using parallel constructs (e.g., OpenMP, MPI).
2. **Automatic parallelization:** Use compilers or libraries that automatically divide work among cores.

c) What is hardware multithreading, and what are its types? Describe their advantages and disadvantages. <L1/CO1>

Answer:

Hardware multithreading allows a processor to execute multiple threads within a single core to improve resource utilization.

Types:

1. **Coarse-grained multithreading:** Switches threads only on long-latency events (e.g., cache misses).
 - *Advantage:* Simple to implement.
 - *Disadvantage:* Idle time during short stalls.
 2. **Fine-grained multithreading:** Switches threads every cycle.
 - *Advantage:* Keeps pipeline busy.
 - *Disadvantage:* Increased complexity and context switching overhead.
 3. **Simultaneous multithreading (SMT):** Executes multiple threads simultaneously in one cycle (e.g., Intel Hyper-Threading).
 - *Advantage:* Maximizes CPU utilization.
 - *Disadvantage:* Shared resources can cause contention.
-

Q3

a) Dr. X (your name) has developed a parallel application where 15% of the code is serial. He uses a PARAM Shavak HPC system (40 cores). Calculate the absolute and maximum possible speedup.

<L1/CO2>

Answer:

Given:

- Serial fraction (f) = 0.15
- Parallel fraction = 0.85
- Number of cores (N) = 40

Using Amdahl's Law:

$$S = \frac{1}{f + \frac{(1-f)}{N}}$$

$$S = \frac{1}{0.15 + \frac{0.85}{40}} = \frac{1}{0.17125} \approx 5.84$$

Absolute Speedup = 5.84

Maximum possible speedup ($N \rightarrow \infty$):

$$S_{max} = \frac{1}{f} = \frac{1}{0.15} = 6.67$$

b) What is the unit to measure computational power of a system? Calculate the computational power of your laptop/node. <L1/CO2>

Answer:

The computational power of a system is measured in **FLOPS (Floating Point Operations Per Second)**.

If a laptop has:

- 8 cores $\times$ 2.5 GHz clock $\times$ 8 FLOPs per cycle

Then,

$$\text{Computational Power} = 8 \times 2.5 \times 10^9 \times 8 = 160 \times 10^9 = 160 \text{GFLOPS}$$

Hence, the system has **~160 GFLOPS** computational power.

Q4

a) Write an OpenMP program to print the names of your family members from different threads along with their job IDs. <L1/CO2>

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main() {  
    char *names[] = {"Father", "Mother", "Brother", "Sister"};  
    #pragma omp parallel num_threads(4)  
    {  
        int id = omp_get_thread_num();  
        printf("Thread %d: %s (Job ID: %d)\n", id, names[id], id);  
    }  
    return 0;  
}
```

b) Write an OpenMP program to print the sum of squares of thread IDs. <L1/CO2>

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main() {  
    int sum = 0;  
    #pragma omp parallel reduction(+:sum)
```

```

{
    int id = omp_get_thread_num();

    int sq = id * id;

    printf("Thread %d: Square = %d\n", id, sq);

    sum += sq;
}

printf("Sum of squares = %d\n", sum);

return 0;
}

```

Q5

a) Write an OpenMP program with a private variable "Aryabhatta = 10" to print the product of thread ID and the variable. <L1/CO2>

```

#include <stdio.h>

#include <omp.h>

int main() {
    int Aryabhatta = 10;

    #pragma omp parallel private(Aryabhatta)
    {
        Aryabhatta = 10;

        int id = omp_get_thread_num();

        printf("Thread %d: Result = %d\n", id, id * Aryabhatta);
    }

    return 0;
}

```

b) Write an OpenMP program to calculate the partial sum of the first 20 natural numbers using lastprivate. <L1/CO2>

```

#include <stdio.h>

#include <omp.h>

```

```

int main() {
    int total = 0;

    #pragma omp parallel for lastprivate(total)
    for (int i = 1; i <= 20; i++) {
        total += i;
    }

    printf("Final Sum = %d\n", total);
    return 0;
}

```

Q6

Parallelizing matrix multiplication using OpenMP (Static vs Dynamic Scheduling) <L1/CO2>

```

#include <stdio.h>

#include <omp.h>

#define N 500

int main() {
    double A[N][N], B[N][N], C[N][N];

    for(int i=0;i<N;i++)
        for(int j=0;j<N;j++)
            A[i][j] = B[i][j] = i + j;

    double start, end;

    // Static Scheduling
    start = omp_get_wtime();

    #pragma omp parallel for schedule(static)
    for(int i=0;i<N;i++)
        for(int j=0;j<N;j++){
            C[i][j]=0;

```

```

        for(int k=0;k<N;k++)
            C[i][j]+=A[i][k]*B[k][j];
    }
end = omp_get_wtime();
printf("Static Schedule Time: %lf sec\n", end - start);

// Dynamic Scheduling
start = omp_get_wtime();
#pragma omp parallel for schedule(dynamic)
for(int i=0;i<N;i++)
    for(int j=0;j<N;j++){
        C[i][j]=0;
        for(int k=0;k<N;k++)
            C[i][j]+=A[i][k]*B[k][j];
    }
end = omp_get_wtime();
printf("Dynamic Schedule Time: %lf sec\n", end - start);

return 0;
}

```

Comparison:

- **Static scheduling:** Assigns fixed iterations per thread — efficient for uniform workloads.
- **Dynamic scheduling:** Assigns iterations at runtime — better for irregular workloads but adds scheduling overhead.